

Libreto del taller

Sistemas Multi-Agente y Function Calling · Hackathon FISC AI AGENTS · UTP, salon 3-303
· dos horas

Como se lee esto

- El **texto negro grande es literal**: es lo que dices en voz alta. Lo que esta en **negrita** dentro de una frase, acentualo.
- Lo *gris y en cursiva* NO se dice: es lo que haces, o la respuesta que estas buscando.
- Lo **azul** son las preguntas que haces al salon. Despues de cada una, **callate** y espera.
- La **franja verde** es cuando ellos trabajan y tu no hablas.
- La barrita gris corta es una pausa. Respira. No la llenes.

De donde salen los 120 minutos. Hablando: 8157 palabras, unos **60 min** a 135 palabras por minuto. Ellos trabajando: **46 min** en las franjas verdes. Queda **14 min**, y no son relleno: son las corridas, la espera del modelo, las preguntas sueltas y algun 429. Hablas el **50%** del taller.

Los relojes de cada bloque son el plan; estas cifras son lo que de verdad hay escrito aqui. Si editas el texto, se recalculan.

Apertura

Buenas. En el taller pasado vieron tres cosas: agentes, herramientas y memoria. Ya saben registrar una funcion con arroba-agente-punto-tool-plain, y ya vieron un agente decidir cual herramienta usar. Hoy vamos a lo que sigue: un agente que manda a otros agentes.

Son dos horas. Y les digo de una lo que se llevan, porque esto no es teoria. Al final van a tener corriendo, en su maquina, un orquestador con subagentes, con salida estructurada, corriendo en paralelo, y que no se cae cuando un hijo se cae. Y el ultimo ejercicio es el esqueleto del MVP que presentan el sabado. No es un ejemplo de juguete que despues botan. Es su punto de partida.

Antes de escribir una linea, la frase del dia. Si se les olvida toda la sintaxis de hoy, esta no: **un subagente es un agente normal, llamado desde dentro de una herramienta de otro agente.**

Eso es todo. No hay una clase especial. No hay framework de orquestacion. No hay nada magico. Es una funcion que por dentro llama a otro agente. Cuando lo vean les va a parecer poca cosa, y esta bien que les parezca poca cosa. Lo dificil no es escribirlo. Lo dificil es saber cuando vale la pena, y eso es la mitad del taller.

Empecemos por el problema, no por el patron. Porque si les enseno el patron primero, lo van a usar en todo, y la mitad de las veces va a estar de mas.

Imaginense un solo agente. Uno. Le dan quince herramientas: leer archivos, escribir, buscar, llamar a una API, consultar la base de datos, mandar correo. Y le dan una tarea que necesita treinta pasos. ¿Que se rompe?

Se rompen cuatro cosas, y las van a ver todas hoy, en este orden.

Una: **el contexto**. Cada paso le reenvia al modelo todo lo anterior. En el paso treinta esta cargando veintinueve pasos de basura, incluidos archivos completos que leyo en el paso tres y que ya no importan. Se vuelve lento, caro, y en algun momento no cabe.

Dos: **las decisiones**. Con quince herramientas, en cada paso el modelo elige entre quince. Se equivoca mas que eligiendo entre tres. Y los errores se acumulan: un paso mal elegido contamina todo lo que viene detras.

Tres: **los permisos**. Si el agente tiene escribir-archivo, la tiene SIEMPRE. En los treinta pasos. Y basta uno raro para que te sobreescriba algo.

Y cuatro: **el costo**. Un solo modelo, normalmente el bueno y el caro, haciendo tanto el razonamiento duro como el trabajo mecanico de listar una carpeta.

Los subagentes son la respuesta a esas cuatro. No a una idea de orden. A esas cuatro. Y esa es la razon por la que hoy vamos a medir cosas en vez de hablar de arquitectura: si no puedo mostrarles el numero, no les estoy dando una razon, les estoy dando una moda.

Y ya que estamos: **multi-agente esta de moda y la mayoría de la gente lo usa mal**. La mitad de los "sistemas multi-agente" que van a ver en internet deberian ser un agente con tres herramientas. Al final del taller les doy un filtro de cuatro preguntas para saber cuando NO hacerlo. Si el sabado el filtro les dice que no hace falta, no lo hagan. Un MVP simple que corre le gana a una arquitectura bonita que se cae en el demo.

Una cosa mas antes de la logistica, para los que llegaron hoy sin haber venido al taller pasado. Un agente, en una frase, es un modelo dentro de un bucle con permiso para llamar funciones. Eso es. El modelo no ejecuta nada: dice "quiero llamar a buscar-texto con este argumento", y su codigo es el que ejecuta y le devuelve el resultado. Todo el poder de un agente son las funciones que ustedes le dieron. Ni una mas.

Y por eso todo el taller de hoy es, en realidad, sobre **que funciones le das, como se las describes, y a quien se las das**. No hay otra palanca.

Dejenme contarles como se ve esto el sabado, para que sepan hacia donde vamos. El patron de MVP que se cae es siempre el mismo: un agente, doce herramientas, un prompt de dos paginas que dice "eres un asistente experto, no cometas errores, no borres archivos". Funciona en la prueba con el caso facil. Y en el demo, con el jurado mirando, elige la herramienta equivocada, o se queda en un bucle, o se come la cuota en el minuto dos.

El que aguanta se ve distinto: pocas herramientas por agente, descripciones aburridas y precisas, limites puestos desde el principio, y respuestas que el codigo puede verificar. Menos impresionante de explicar, mucho mas dificil de romper. Eso es lo que vamos a construir hoy, en seis pasos.

Ahora, logistica, y pongan atencion aqui porque de esto depende que lleguen al final del taller con cuota.

Hay dos modos de trabajo. MODO igual test no usa red ni llave: usa el TestModel de Pydantic AI, que llama a todas sus herramientas con datos de relleno y no razona. Sirve para comprobar que su codigo corre. Y MODO igual gemini da respuestas de verdad y necesita la llave.

La regla es: **armen en test, prueben en gemini**. Escriban, verifiquen la forma en test, y solo cuando ya corre, cambien a gemini para ver el comportamiento real. Porque la capa gratis de Google da entre cinco y quince peticiones por minuto. Si prueban cada cambio contra el modelo real, a la mitad del taller se quedan sin cuota y terminan mirando la pantalla del vecino.

Y cada quien necesita SU llave. No la compartan. Una llave entre treinta personas se muere en el primer ejercicio, y ahi el problema deja de ser de Google y pasa a ser mio.

¿A quien le falta la llave? Levanten la mano.

Los que levanten: díles que trabajen en MODO=test todo el taller, que funciona completo, y que saquen la llave en aistudio.google.com mientras corren el ejercicio 1. No los dejes bloqueados esperando.

[Escribe en el pizarron las dos lineas del .env. Las van a pedir cuatro veces.]

Si algo no les corre, lo primero es python verificar punto py. Les dice exactamente que falta. Y si les sale un cuatrocientos veintinueve en cualquier momento del taller: es el limite de tasa de Google, no es su codigo. El repo se los dice con todas sus letras. Esperan un minuto o se pasan a test y siguen.

10:00 – 28:00

Ejercicio 1 · la descripcion es el prompt

Ejercicio uno. Abran ejercicios, cero-uno, herramientas punto py, y corranlo. Sin leerlo. Primero corre, despues lee.

[Proyecta tu propia corrida mientras ellos corren la suya.]

Lo que acaban de ver es un agente con una herramienta que se llama buscar-texto. Y su docstring dice, completo: "Busca cosas." Dos palabras.

Fijense en lo que paso. No dio error. No se cayo. Hizo una llamada equivocada con total confianza. Y eso es lo peligroso, porque un error lo ves; una respuesta segura pero mal fundamentada te la crees.

Aqui esta la idea del ejercicio, y es la que mas les va a servir el sabado: **el modelo no ve su codigo**. No lee el cuerpo de la funcion. Nunca. Ve tres cosas, nada mas. El nombre de la funcion. Los tipos de los parametros. Y el docstring. Esa es toda la interfaz.

O sea que su docstring no es documentacion. **Es prompt**. Es la unica instruccion que el modelo tiene para decidir si usa esa herramienta, cuando, y con que argumentos. Y si escriben un prompt de dos palabras, van a tener el comportamiento que merece un prompt de dos palabras.

Las tres partes cuentan, no solo el docstring. El **nombre** es lo primero que el modelo lee: buscar-texto le dice algo; func-dos o helper-final no le dicen nada. Los **tipos** son restricciones reales: si ponen linea dos puntos int, el modelo no les va a mandar "cuarenta y dos" en letras, porque Pydantic no se lo permite. Los tipos no son adorno, son la primera validacion.

Ahora arreglen ese docstring. Y un docstring que sirve dice tres cosas.

Uno: **que devuelve, y en que formato**. No "devuelve los resultados". Devuelve lineas con formato archivo, dos puntos, numero, dos puntos, contenido, maximo veinte. Asi el modelo sabe que va a recibir antes de llamarla, y sabe como leerlo cuando llegue.

Dos: **que NO hace, o que no acepta**. Esta es la que evita la mitad de las llamadas malas. Si su herramienta no acepta carpetas, diganlo. Si no entiende expresiones regulares, diganlo. Porque si no lo dicen, el modelo lo va a intentar — no por tonto, sino porque nada se lo prohibio.

Y tres, la que nadie escribe y la que mas cuesta: **que significa el resultado vacio**. Si "Sin coincidencias" significa "no esta", y no significa "la llamada fallo", diganlo explicitamente. Porque si no, el modelo asume que fallo, y reintenta. Y vuelve a reintentar. Y ahi se les va la cuota, en un bucle que ustedes causaron con una omision de seis palabras.

Ese es el patron que quiero que se lleven: **los agentes no fallan tanto por lo que les dices; fallan por lo que no les dijiste**.

Y ya que estamos en herramientas, tres decisiones de diseno que van a tener que tomar el sabado y que casi nadie les va a explicar.

La primera: **que tan grande hacen cada herramienta**. Tienen dos extremos. Una sola herramienta gigante que hace todo — buscar, leer y resumir — o tres chiquitas. La grande le quita decisiones al modelo, y eso suena bien, pero tambien le quita informacion: si falla, no sabes en que parte fallo, y el modelo tampoco. Las chiquitas le dan control y te dan visibilidad. La regla practica: **una herramienta, un verbo**. Si el nombre de su funcion necesita un "y", probablemente son dos.

La segunda: **cuanto devuelven**. Miren leer-archivo en el repo: corta a cuatro mil caracteres. No es pereza, es presupuesto. Si una herramienta devuelve un archivo de cincuenta mil caracteres, eso entra al historial, y a partir de ahi **se reenvia en cada vuelta** del bucle. Una sola llamada descuidada les puede triplicar el costo de toda la corrida. Devolver de mas es tan malo como devolver de menos, y es mas caro.

La tercera: **que las llamen dos veces sin miedo**. Los agentes reintentan. Se equivocan y repiten. Si su herramienta manda un correo o cobra una tarjeta, que la llamen dos veces es un problema de verdad, no un detalle. En el sandbox de hoy todo es leer y escribir archivos y no pasa nada. El sabado, si le dan al agente algo que toca el mundo real, esa pregunta la tienen que contestar antes.

Fijense que ninguna de las tres es sobre el prompt. Son de diseno. El prompt del ejercicio uno se arregla en un minuto; estas tres se arreglan cuando decides la forma de la herramienta.

Y una cosa mas sobre los tipos, porque es la validacion mas barata que existe y casi nadie la usa a fondo.

Cuando ustedes escriben ruta dos puntos str, Pydantic AI le manda al modelo un esquema que dice "este parametro es texto". Si escriben linea dos puntos int, el modelo no puede mandarles "cuarenta y dos" en letras: no pasa la validacion. Eso es una restriccion que ustedes consiguieron gratis, sin escribir una sola linea de comprobacion.

Y se puede apretar mucho mas. En vez de un string libre para un modo, usen un tipo cerrado con las tres opciones validas. En vez de un int suelto, un int con minimo y maximo. Cada vez que aprietan el tipo, le quitan al modelo una manera de equivocarse.

La forma de pensarlo es esta: **el docstring es persuasion, el tipo es ley**. El docstring le explica al modelo lo que deberia hacer y normalmente hace caso. El tipo decide lo que **puede** llegar a su codigo. Y es la misma idea que vamos a repetir tres veces hoy: si algo tiene que cumplirse, no lo pidan. Hagan que sea imposible incumplirlo.

ELLOS TRABAJAN — tu te callas. 9 min. Arreglan el docstring y escriben su propia herramienta desde cero. Circula. A quien acabe temprano, pidele que lea su docstring en voz alta; se aprende mas del de al lado que de ti.

Una ultima cosa de este ejercicio, y haganla ahora mismo: pidanle un archivo que no existe.

El agente leyó el error y reintento. No se murió. Y eso no es suerte: es una decisión de diseño, y está en taller, herramientas punto py. Todas esas funciones devuelven el error **como texto** en vez de lanzar excepción.

Piensenlo desde el agente. Si la función revienta, la excepción sube y mata la corrida: el agente no se enteró de nada, simplemente dejó de existir. Si la función devuelve "ERROR: el archivo no existe", eso entra al historial como un mensaje más, el modelo lo LEE, y corrige en el siguiente paso.

Apuntense esto para el sábado, porque es de las cosas más útiles del taller: **en las herramientas de un agente, un error no es una excepción. Es un dato.** Y lo vamos a volver a usar en el ejercicio cinco, con subagentes enteros en vez de funciones.

28:00 – 42:00

Ejercicio 2 · que hace run_sync por debajo

Ejercicio dos. Es el más corto de escribir — no tiene nada que escribir — y es el que más les va a servir cuando algo falle el sábado a las dos de la mañana.

[Proyecta el bucle y déjalo en pantalla todo el bloque.]

run-sync no es magia. Es este bucle, y son seis líneas. Armas una lista de mensajes con el objetivo. Le pasas la lista al modelo. Si el modelo no pide herramientas, terminaste. Si pide, ejecutas la herramienta, le pegas la salida a la lista, y vuelves a llamar al modelo con la lista completa. Y otra vez.

Fíjense en la línea que dice mensajes punto append. Sin esa línea el bucle no avanza nunca: el modelo volvería a pedir la misma herramienta eternamente, porque nadie le dijo el resultado.

De ahí salen dos cosas, y las dos importan más de lo que parecen.

La primera: **el modelo no tiene memoria**. Ninguna. Cero. No "recuerda" el paso anterior. En cada vuelta se le reenvia el historial entero, desde el principio. Esa lista de mensajes ES la memoria. No hay nada mas. Cuando alguien dice que su agente "se acuerda", lo que pasa es que alguien esta reenviando texto.

La segunda sale de la primera, y es de plata. El paso uno manda un mensaje. El paso dos manda tres. El paso diez manda los nueve anteriores mas el nuevo. O sea que el costo de una corrida **no crece con los pasos: crece mas rapido que los pasos**.

Hagan la cuenta gruesa conmigo. Digamos que cada paso agrega mil tokens. En el paso diez no pagaron diez mil: pagaron mil, mas dos mil, mas tres mil... hasta diez mil. Cincuenta y cinco mil tokens para diez pasos que "generaron" diez mil. Y en treinta pasos son casi medio millon.

Y hay un segundo techo, que no es de plata: la **ventana de contexto**. En algun punto el historial ya no cabe en la peticion. Ahi no te sale una factura mas alta: te sale un error, o el modelo empieza a olvidar el principio. Los subagentes del proximo ejercicio existen sobre todo por esto.

Antes de que corran, les explico que van a ver, porque el archivo imprime el historial completo y a la primera parece sopa de letras.

Van a ver dos tipos de mensaje alternandose. **ModelRequest** es lo que ustedes le mandaron al modelo. **ModelResponse** es lo que contesto. Y adentro de cada uno hay partes: una ToolCallPart es el modelo pidiendo una herramienta, y una ToolReturnPart es el resultado que su codigo le devolvio.

Cuenten los ModelResponse. **Cada ModelResponse es una peticion que pagaron**. Y si hay cuatro, es porque el bucle dio cuatro vueltas, no porque hicieron cuatro cosas. Esta es la pantalla que van a querer mirar el sabado cuando su agente "se tarde mucho" o "gaste mucho": casi siempre la respuesta esta ahi, en una vuelta de mas que nadie pidio.

ELLOS TRABAJAN — tu te callas. 4 min. Corren y miran el historial impreso. Que cuenten los ModelResponse. Despues bajan request_limit a 2 en taller/config.py y corren otra vez.

Lo que acaban de ver con el limite en dos es UsageLimits haciendo su trabajo: corto el agente antes de que siguiera gastando.

Y hay dos topes distintos, no uno. **request-limit** cuenta llamadas al modelo: te protege del bucle de razonamiento, del agente que piensa en circulos. **tool-calls-limit** cuenta ejecuciones de herramientas: te protege del agente que lee cuatrocientos archivos porque no encontro lo que buscaba. Son fallos distintos y hacen falta los dos.

Ese par de numeros es lo unico que separa un bug de una factura. Con cuota gratis no es higiene, es supervivencia. **Ponganlos antes de la primera corrida, no despues del susto.**

Y una pregunta que siempre sale aqui: si el problema es que el historial crece, ¿por que no lo cortamos y ya? Se puede, y hay tres maneras, con sus costos.

Una: **truncar**. Botan los mensajes viejos. Es gratis y es peligroso: si botan el que tenia el dato que hacia falta, el agente se vuelve a poner a buscarlo, y gastan mas de lo que ahorraron.

Dos: **resumir**. Cada tantos pasos, le piden al modelo que comprima lo anterior. Funciona, pero cuesta una peticion, y cada resumen pierde detalle. Resumen de resumen de resumen y ya no queda nada concreto.

Y tres: **no meterlo nunca**. Que el trabajo sucio pase en otro historial, y que al principal solo llegue la conclusion. Eso es un subagente, y es la unica de las tres que no pierde nada del historial principal, porque nunca lo ensucio.

Si su MVP necesita treinta pasos, ¿que le pasa al historial? ¿Y al costo? ¿Y a la ventana de contexto?

Dejalos contestar. NO contestes tu. La respuesta es la puerta al ejercicio 3; si la das tu, el ejercicio 3 pierde el porque.

Exacto. Y esa es la razon por la que existe el ejercicio tres.

Y antes de pasar, dos minutos sobre algo que no esta en ningun tutorial y que el sabado les va a salvar la noche: **como se depura un agente**.

El problema es que un agente que falla casi nunca tira un error. Les devuelve una respuesta razonable que esta mal. Y entonces la gente hace lo unico que se le ocurre: le cambia el prompt. Le agrega "por favor se preciso", lo corre otra vez, sale distinto, y creen que lo arreglaron. No arreglaron nada: movieron la ruleta.

Hay tres cosas que mirar, y las tres estan en este repo.

La primera: **el historial**. El que acabamos de imprimir. Ahi se ve que herramienta pidio, con que argumentos, y que le devolvio. El noventa por ciento de los "el modelo esta tonto" son en realidad un argumento mal armado que esta escrito ahi, en texto plano, esperando que alguien lo lea.

La segunda: **el uso**. Peticiones y herramientas. Si esperaban tres pasos y gastaron nueve, hubo un bucle. Ese numero les dice que hay un problema antes de que la factura se los diga.

Y la tercera, que es una costumbre mas que una herramienta: **impriman lo que le mandan a cada subagente**. Fijense que las delegaciones del ejercicio tres imprimen "delegando al explorador". En su MVP, impriman tambien **el encargo**. Una linea. Y cuando algo salga raro, van a ver con sus ojos lo que el hijo recibio en vez de suponerlo. Van a entender por que insisto en esto en unos veinte minutos.

42:00 – 72:00

Ejercicio 3 · el bucle multi-agente

Ejercicio tres. Este es el del taller. Si hoy solo se llevan un archivo, que sea este.

[Proyecta el patron de cinco lineas y dejalo en pantalla.]

Miren el patron. Son cinco lineas. Es una herramienta del orquestador, decorada con arroba-orquestador-punto-tool. Y por dentro, en vez de hacer el trabajo, llama a otro agente y devuelve su salida. Nada mas.

Una diferencia de sintaxis que importa: aquí es punto-tool, no punto-tool-plain. La version sin plain te pasa un contexto como primer parametro, el ctx. Lo necesitan para una cosa que vamos a ver en un rato, y es la que mas gente olvida.

Y aquí está lo importante: **para el orquestador esto es una herramienta cualquiera**. No sabe que por dentro corrió otro agente con su propio bucle de diez pasos. Ve una función que le devolvió un texto. Igualito que buscar-texto en el ejercicio uno.

Ahora, las cuatro razones. Son las cuatro cosas que se rompían con el agente único de hace media hora, en el mismo orden.

La primera, y la principal: **aislamiento de contexto**. El subagente puede quemar cuarenta mil tokens leyendo archivos, y devolverle doscientas palabras al padre. Esos cuarenta mil **nunca entran** al historial del orquestador.

Y acuerdense de la cuenta del ejercicio dos: cada paso del padre reenvía todo lo suyo. Si el padre cargara todo lo que leyeron sus hijos, reventaría en tres pasos. Por eso el aislamiento no es orden ni elegancia: es lo que hace que la cosa **funcione** a treinta pasos.

La segunda: **menos herramientas por decision**. El orquestador elige entre dos: explorar o escribir. El explorador elige entre tres. Nadie elige entre quince. Y menos opciones por paso es menos errores por paso.

La tercera: **modelo por tarea**. El orquestador razona, y puede necesitar el modelo bueno. El que lista una carpeta y copia texto no necesita razonar nada, y puede correr en el más barato y más rápido. Hoy los dos corren en el mismo porque estamos en cuota gratis, pero el sábado eso es plata real.

Y la cuarta: **permisos**. Esta mírenla en el código, porque es la que más les va a importar el sábado.

[Enseña al explorador: sus tres herramientas, y que no está escribir_archivo.]

El explorador tiene tres herramientas: buscar, leer y listar. No tiene escribir-archivo. Y no es que le pedimos amablemente que no escriba. **No tiene con que.** Aunque el modelo decida que lo mejor del mundo seria sobrescribir un archivo, no hay funcion que llamar.

Digan esto conmigo, porque es la idea mas importante del taller despues de la del principio: **un prompt que dice "no borres nada" es una recomendacion. Una funcion que no existe es una garantia.**

[Abre taller/herramientas.py y ensena _ruta_segura(). Son seis lineas.]

Lo mismo con el sandbox. Ningun agente de este repo puede tocar un archivo fuera de la carpeta sandbox. Y no es porque el prompt lo pida: es porque hay una funcion que resuelve la ruta y la rechaza si se sale. Seis lineas de Python.

Y fijen en el detalle, que es donde casi todo el mundo se equivoca: compara **rutas**, no texto. Si compararan el texto del principio, una carpeta hermana llamada sandbox-guion-malo pasaria el filtro, porque su ruta empieza igual que la del sandbox. Esa clase de error es exactamente la que uno no ve hasta que alguien la busca.

Esa es la idea general: **lo que garantiza algo es el codigo, no el prompt.** El prompt es para que se porte bien. El codigo es para cuando no.

Ahora, la pregunta que de verdad importa y que nadie contesta: ¿**donde ponen la linea?** ¿Como deciden que dos agentes y no uno, o cuatro?

Mi respuesta practica: **corten donde el contexto se puede tirar.** Si despues de un pedazo de trabajo se puede botar todo lo que se leyo y quedarse solo con la conclusion, ahi hay un subagente. Si el siguiente paso necesita todo el detalle del anterior, ahi no hay frontera y partirlo solo les agrega una traduccion.

Miren nuestro caso con ese lente. El explorador lee tres archivos completos y de todo eso lo unico que hace falta despues es "el bug es este". Se puede tirar el resto. Frontera clarisima. El escritor, en cambio, recibe el texto y lo escribe: no genera contexto que valga la pena tirar. ¿Y por que lo separamos, entonces? No por contexto. **Por permisos.** Es el unico que tiene escribir-archivo. Razon distinta, misma decision.

Y una que les va a picar: ¿puede un subagente tener sus propios subagentes? Si. Tecnicamente no hay nada que lo impida. Y para el sabado, mi consejo es: **no pasen de un nivel.** Cada nivel es otra traduccion, y ya van a ver en un rato lo que cuesta una sola. Dos niveles de delegacion en un MVP de hackathon es casi siempre una forma elegante de no saber por que salio mal.

Otra cosa, y tomenle foto al codigo porque es facil de pasar por alto: miren las instrucciones del explorador. Dicen "maximo ochenta palabras" y le dan una estructura.

¿Por que? Porque **la respuesta del subagente es lo unico que entra al padre.** Si el hijo contesta con quinientas palabras, acabamos de perder la mitad del aislamiento que nos costo tanto conseguir. El limite de largo del hijo no es cortesia: es la puerta por donde entra el costo al padre. Ponganselo a todos sus subagentes.

Y al reves, en el orquestador: sus instrucciones dicen que no investiga ni escribe el mismo, que delega, y en que orden. Un orquestador sin esa frase tiende a intentar hacer el trabajo solo, y entonces no tienen un orquestador: tienen un agente con pasos extra.

Y el sintoma numero uno con el que van a pelear el sabado: **el orquestador no llama a su subagente.** Simplemente no lo usa, contesta cualquier cosa y termina. Van a pensar que el subagente esta roto.

Casi nunca esta roto. Revisen en este orden. Uno: el **docstring** de la herramienta — ¿dice cuando usarla? Si dice "explora cosas", estamos en el ejercicio uno otra vez. Dos: las **instrucciones del orquestador** — ¿le dijeron que delegue? Tres: ¿se quedo sin **limite** antes de llegar? Y cuatro, ya de ultimo, el subagente.

Ese orden les va a ahorrar horas. El problema casi siempre esta del lado del que decide, no del que ejecuta.

Y el sintoma numero dos, que es el contrario: **el orquestador no para**. Delega, le devuelven algo, y en vez de contestar vuelve a delegar. Otra vez. Hasta que se acaba el limite.

Eso pasa porque nadie le dijo **cuando terminar**. Y no hay una funcion para eso: es una frase en sus instrucciones. Digan explicitamente que hacer cuando ya tenga la informacion: "cuando el explorador te devuelva el hallazgo, escribe tu respuesta final y detente". Sin eso, un modelo servicial va a seguir buscando mas contexto para siempre, porque mas contexto siempre parece mejor.

Y ahi es donde el limite del ejercicio dos se gana el sueldo: cuando el prompt falla, el tope corta. El prompt es la intencion; el limite es la red. Necesitan los dos, y por razones distintas.

Ahora les toca, y es la tarea grande del dia. Van a escribir el segundo subagente: el escritor.

Los TODO en el archivo les dicen que tiene que cumplir, no como se escribe. Eso es a proposito — no les puse el codigo comentado para que lo descomenten. El explorador que esta justo arriba es su ejemplo, y esta funcionando: copien la forma, no el texto.

Tres pasos. Uno: el Agent, con sus instrucciones. Diganle que hace, que NO hace, y cuanto mide su respuesta. Dos: su unica herramienta, escribir-archivo. Solo esa — si de paso le dan leer-archivo, deja de ser un escritor y el aislamiento de permisos se les cae, que es lo que acabamos de decir que importaba. Y tres: registrenlo como herramienta del orquestador, con tool, no con tool-plain.

ELLOS TRABAJAN — tu te callas. 12 min. Es la tarea principal del taller. Si alguien lleva mas de cinco minutos trabado en un typo, que abra soluciones/ y siga con el grupo: no se pierde veinte minutos ahi.

Ahora cambien el OBJETIVO a algo que necesite los dos — averiguar el bug y escribir un resumen en reporte punto md — corran, y abran el archivo.

Y fijense en una cosa del sandbox, porque no es casualidad: **el bug esta repartido**. Notas punto txt dice el sintoma: el login falla cuando el correo tiene mayusculas, ticket cuatrocientos doce. Y docs, arquitectura punto md, dice la causa: el correo se guarda tal cual, sin normalizar.

No se puede contestar leyendo un solo archivo. Eso es lo que justifica tener un explorador: si la respuesta estuviera en un archivo, esto seria una herramienta, no un subagente.

Miren ahora el reporte del final. Dice cuantos tokens de entrada quemo cada subagente, y cuantos caracteres devolvio. Esa segunda columna es lo unico que entro al historial del orquestador. Lo de la primera se quedo adentro del hijo, y nadie lo paga dos veces.

En MODO=test la relacion sale AL REVES: TestModel repite la salida de las herramientas en vez de resumir, y el explorador devuelve mas de lo que quemo. El programa lo avisa solo. Si quieres que se vea de verdad, proyecta TU corrida en MODO=gemini: es el mejor momento del taller para gastar cuota en publico.

Una cosa mas, y es la que olvida todo el mundo. Quiten usage igual ctx punto usage de una delegacion, y corran.

¿Que le paso al conteo? El gasto del hijo dejo de contar para el padre. Y eso suena inofensivo hasta que lo dicen al revés: si el gasto del hijo no cuenta, **el limite del padre no esta limitando nada**. Ustedes creen que pusieron un techo de doce peticiones, y en realidad pusieron doce para el padre y ninguno para los hijos. Tres subagentes sueltos son tres presupuestos sin techo. Por eso necesitaban el ctx.

Y fijense en la consecuencia, que esta en config punto py: hay dos juegos de limites. LIMITES, para un agente solo, con seis peticiones. Y LIMITES-ORQ, para el orquestador, con doce. **No es que el orquestador sea mas importante: es que su cuenta incluye a sus hijos**. Si delegan tres veces y cada hijo da tres vueltas, el padre ya lleva nueve peticiones sin haber pensado casi nada el mismo.

O sea que cuando agreguen un subagente al MVP, tienen que subir el techo del padre. Y si no lo suben, el sintoma que van a ver no es "me quede sin limite": es que **el ultimo subagente nunca corre**, porque la cuota del padre se acabo antes de llegar a el. Ese es de los bugs mas confusos que hay, porque el codigo de ese subagente esta perfecto.

Y ahora la ultima tarea de este bloque, que es la que menos se ensena en ningun lado. Van a romper algo que no da error.

En la herramienta explorar, antes de llamar al subagente, ignoren lo que les pidio el orquestador y mandenle un encargo vago: "Dime que hay en el proyecto." Corran con el objetivo de antes, y abran reporte punto md.

ELLOS TRABAJAN — tu te callas. 3 min. Que lo hagan y que abran el reporte.

Veán lo que paso. El explorador **no fallo**. Contesto bien. Contesto perfectamente una pregunta que no era la que hacia falta. Y el orquestador tomo esa respuesta y escribio su reporte igual de convencido. Cero errores. Ningun cuatrocientos veintinueve. Ningun limite alcanzado. Y el resultado esta mal.

Ese es el costo real de delegar, y no es el que la gente cuenta. No son los tokens del salto. No es la latencia. Es que **el padre le encarga al hijo algo peor de lo que cree**. Cada delegacion es una traduccion, y en cada traduccion se pierde algo.

Guardense esto para el sabado: cuando su MVP falle y los dos agentes "funcionen" — los dos corren, ninguno tira excepcion, los logs estan limpios — **miren el encargo primero**. No el prompt del hijo, no el del padre: lo que el padre le paso al hijo.

Y ya que les ensene como se rompe, les digo como se arregla, porque si no esto es solo una anecdota triste.

Un buen encargo lleva tres cosas. La primera: **el objetivo original, no la version resumida del padre**. Suena obvio y es el error mas comun: el orquestador "interpreta" lo que le pidieron y le manda al hijo su interpretacion. Si el objetivo era "averigua el bug y escribe un resumen", el explorador necesita saber que hay un resumen al final, porque eso cambia lo que le conviene recoger.

La segunda: **que va a hacer el padre con la respuesta**. "Necesito esto para escribir un reporte de una frase" produce una respuesta distinta a "dime todo lo que encuentres". El hijo no puede optimizar lo que no sabe.

Y la tercera: **que NO hace falta**. Igual que en los docstrings del ejercicio uno. "No me listes todos los archivos, solo los que tengan que ver con el login." Sin eso, el hijo hace lo razonable, que es traerte todo, y ahi se te va el aislamiento.

Fijense que es el ejercicio uno otra vez, un nivel arriba. Ahi el docstring era el prompt de la herramienta. Aqui **el encargo es el prompt del subagente**. Y las dos cosas las escribe el padre, en tiempo de ejecucion, sin que nadie las revise.

Ejercicio 4 · la salida estructurada

Ejercicio cuatro. Y esto es lo que mas les va a cambiar el MVP por linea escrita, asi que aunque se lleven medio taller, llevense esto.

Hasta ahora nuestro explorador devolvía **prosa**. Ochenta palabras en español. Y el orquestador tenía que leerlas y creerselas. No podía comprobar nada. Piensenlo: "no encuentre la causa" y "la causa es que no se normaliza el correo" son los dos un string. Para distinguirlos hay que entender español.

[Proyecta la clase Hallazgo.]

Aquí declaramos un esquema: una clase que hereda de BaseModel, con cuatro campos. Archivo, un string. Linea, un entero. Sintoma, un string. Y causa, que puede ser un string o puede ser nulo. Y después le pasamos output-type igual Hallazgo al agente.

Con eso, resultado punto output ya no es un string. **Es un Hallazgo**, validado por Pydantic antes de que su código lo vea.

Para que se vea la diferencia, pensemos en lo que el orquestador tendría que hacer con prosa. Le llega: "Encuentre que el login falla con mayusculas, esta reportado en notas punto txt como ticket cuatrocientos doce, aunque no pude confirmar la causa exacta."

Ahora ustedes, desde el código del padre, quieren saber tres cosas: ¿hay causa o no? ¿en que archivo? ¿en que linea? Con ese párrafo tienen tres opciones, y las tres son malas. Escriben expresiones regulares sobre español libre. O le piden a otro modelo que lo interprete, y pagan otra petición para entender la respuesta que ya pagaron. O — y esto es lo que hace todo el mundo — no comprueban nada, le pasan el párrafo al siguiente paso, y rezan.

La tercera opción es la que produce los MVP que funcionan en la prueba y se caen en el demo. Porque funcionan mientras el modelo conteste parecido a como contestaba ayer.

Cuatro cosas cambian, y la primera es la grande.

Una: **el padre puede comprobar**. Si no hallazgo punto causa, dos puntos. Es una línea de Python. El orquestador deja de creer y empieza a verificar. Con prosa, para saber si el hijo encontró la causa, tendrían que preguntarle a otro modelo — o sea, pagar otra petición para interpretar la respuesta que ya pagaron.

Dos: **el modelo ya no elige el formato**. Sin output-type, ustedes le piden en el prompt "responde con archivo, línea y causa", y a veces obedece, y a veces les mete una introducción amable antes. Con output-type no es una petición: si la respuesta no encaja en el esquema, Pydantic AI lo hace **reintentar**. El formato deja de ser suerte.

Tres: **se acaban los parseos**. Nadie tiene que sacar un número de línea de una frase con una expresión regular a las dos de la mañana. Y les prometo que eso, a las dos de la mañana, no sale bien.

Y cuatro: **es más barato**. Un objeto de cuatro campos ocupa menos tokens en el historial del padre que ochenta palabras de prosa. O sea que refuerza el aislamiento del ejercicio tres.

Y hay una cosa que me gusta especialmente de esto, que es que **el esquema ES la instrucción**. Cuando agreguen un campo — por ejemplo confianza, de cero a diez — no van a tocar el prompt, y el modelo lo va a llenar igual. El esquema se le manda como parte de la definición. Es la misma lección del ejercicio uno: la interfaz es el prompt. Antes era el docstring; ahora es el tipo.

Su tarea principal aquí es la comprobación. El docstring de la herramienta ya le promete al orquestador que va a devolver algo que empiece con INCOMPLETO si falta la causa. Cumplanlo: si el hallazgo viene sin causa, no sigan, devuelvan que falta, y dejen que el orquestador decida. Son dos líneas.

Dos detalles del esquema que valen oro y que se ven en tres minutos.

El primero: **los campos opcionales son una decision de producto.**

Fijense que causa puede ser nulo, a proposito, y que las instrucciones le dicen "si no la encuentras, dejala en null, no la inventes". Si hubieramos puesto causa como obligatoria, el modelo **tiene** que llenarla — y cuando un modelo tiene que llenar un campo que no sabe, se lo invents. Ustedes convirtieron un campo obligatorio en una alucinacion garantizada. Lo opcional es lo que le da permiso de no saber.

El segundo: para clasificar, usen tipos cerrados. Un Literal o un Enum — alto, medio, bajo — en vez de un string libre. Con string libre les va a llegar "alto", "Alto", "muy alto" y "high", y van a terminar escribiendo un if con cuatro casos. Con un tipo cerrado, o llega uno de los tres o Pydantic AI lo hace reintentar.

Y la letra chica, para que no los sorprenda: **los reintentos cuestan.** Cada vez que la respuesta no encaja en el esquema, hay otra peticion. Un esquema con quince campos obligatorios y descripciones vagas puede reintentar tres veces por llamada. O sea que un esquema demasiado exigente se les convierte en una factura. Pidan lo que necesitan para decidir. Nada mas.

ELLOS TRABAJAN — tu te callas. 7 min. Escriben la comprobacion y agregan un campo al esquema. En MODO=test la causa sale nula, así que la rama de INCOMPLETO se ve sin gastar llave: eso es una ventaja, no un problema.

Y para que quede claro cuando NO usar esto: si lo que quieren del subagente es de verdad un resumen libre, para que lo lea una persona, dejenlo en prosa. El esquema es para lo que **alimenta una decision de codigo**. La regla practica: si el padre va a hacer un if con la respuesta, que sea un objeto.

Ejercicio 5 · en paralelo, y cuando algo falla

Ejercicio cinco. Dos cosas que separan un demo que aguanta de uno que se cae en el escenario. Y la segunda les va a pasar el sabado, se lo garantizo.

Primera: en serie contra en paralelo. En el ejercicio tres los subagentes corrian uno detras del otro, y estaba bien, porque el escritor necesitaba lo que encontro el explorador. Habia dependencia.

Pero cuando dos subagentes **no dependen uno del otro**, esperar al primero para arrancar el segundo es tiempo regalado. Aqui tenemos dos exploradores: uno busca el sintoma en las notas, el otro busca la causa en docs. Ninguno necesita lo del otro. Se arregla con una linea: `asyncio punto gather`.

Le pasan las dos corrutinas — y fijense, sin `await` en cada una — y `gather` las espera a las dos. La corrida cuesta lo que el mas lento, no la suma de los dos.

Y ahora la letra chica, que es importante con cuota gratis: **el reloj baja, el gasto no**. Son las mismas dos peticiones. Paralelizar no les ahorra ni un token; les ahorra segundos. Para un demo de tres minutos delante de un jurado, esos segundos valen. Para su cuota, no cambian nada. No confundan las dos cosas.

Y hay una trampa con el paralelo que tienen que oír hoy, porque con cuota gratis les va a morder: **el paralelo hace que el limite de tasa sea peor, no mejor**.

Piensenlo. En serie, sus dos peticiones estan separadas por unos segundos. En paralelo, salen las dos **en el mismo instante**. Si Google les da cinco por minuto, dos rafagas simultaneas se acercan mucho mas rapido al techo que las mismas dos espaciadas. Ganaron segundos de reloj y compraron mas probabilidad de un cuatrocientos veintinueve.

Así que el consejo para el sábado es concreto: **paralelicen de dos en dos, no de diez en diez**. Si tienen cinco subagentes independientes y los lanzan todos a la vez con cuota gratis, el gather les va a devolver cinco errores de límite en lugar de cinco respuestas. Y va a ser más lento que haberlos hecho en serie, porque van a tener que reintentarlos todos.

Y la otra cosa que el paralelo no perdona: **si dos subagentes escriben, no los paralelicen**. Dos escritores sobre el mismo archivo, arrancando al mismo tiempo, es una carrera, y el que gana es el que termine último. En paralelo van los que **leen**. Los que escriben, en serie, y con la escritura en un solo sitio.

Segunda cosa, y la más útil: **los subagentes se caen**.

La herramienta del orquestador que escribieron en el ejercicio tres no atrapa nada. Si el hijo lanza una excepción, sube, y se muere el padre, y se muere la corrida completa. Y el sábado un subagente se va a caer: un cuatrocientos veintinueve, una ruta mala, un timeout, una respuesta que no encaja en el esquema.

Y la solución ya la saben, porque la vieron en el ejercicio uno: **devolver el error como texto en vez de lanzarlo**. Ahí era una función que devolvía "ERROR: el archivo no existe". Aquí es un subagente entero. La idea es la misma: si el orquestador lo LEE, decide. Si le explota, no decide nada.

Y miren las instrucciones del orquestador de este archivo: le decimos explícitamente que si una herramienta devuelve algo que empieza con ERROR, no se detenga, use lo que si tiene, y diga que parte falta. Sin esa frase, atrapar la excepción no sirve de nada: el padre se queda con un error en la mano y sin saber que hacer con él.

Hay un detalle en el que van a tropezar, y les aviso antes: tienen que proteger **las dos** delegaciones, la de en-serie y la de en-paralelo. Una sola delegación sin try-except basta para matar la corrida, y el orquestador llama primero a en-serie. Si solo protegen una, van a jurar que su código no funciona.

Y esa es la regla general, no un detalle de este archivo: **toda delegacion va protegida**. No la que creen que es riesgosa. Todas.

Y aprovechen que estamos aqui para pensar en todo lo que se puede caer, porque no es solo una excepcion de Python.

Se puede caer el **limite de tasa** a mitad de corrida: el paso uno pasa y el paso cuatro les da cuatrocientos veintinueve. Se puede caer un **timeout**, que es el peor porque no falla rapido: se queda ahi mientras el jurado los mira. Se puede caer la **validacion** del esquema del ejercicio anterior, despues de gastar los reintentos. Y se puede acabar el **limite** que ustedes mismos pusieron.

Y para cada uno hay dos respuestas posibles, y son distintas: **reintentar**, o **rendirse bien**. Un cuatrocientos veintinueve se reintentan, con una espera. Una ruta mala no: reintentarla da el mismo error y les quema dos peticiones en vez de una. La regla: **reintenta lo que es temporal; rindete con lo que es permanente**. Y si se rinden, que sea devolviendo texto que el padre pueda leer.

Lo cual nos lleva a la decision que de verdad importa, y que no es de codigo: ¿una respuesta a medias sirve o no sirve? Si su MVP encontro el sintoma pero no la causa, ¿lo muestran con una nota, o dicen que no pudieron? Eso es una decision de producto, y va en el prompt del orquestador. **Si no se la dicen, se la inventa**.

Mi consejo para el sabado: que su MVP prefiera **siempre** decir "esto es lo que tengo, y esto me falta" antes que fallar en silencio o inventar. Un jurado perdona un resultado incompleto que se anuncia. No perdona uno que se descubre en vivo.

ELLOS TRABAJAN — tu te callas. 6 min. Tarea 2: gather. Tareas 3 y 4: descomentan el raise, ven morir la corrida, y despues la protegen. Recuerdales que son las dos delegaciones.

Y ya que hablamos del demo, les dejo la lista de los diez minutos antes de presentar. Cinco cosas. Apuntenlas.

Una: **corranlo completo, una vez, antes de subir.** No un pedazo: de punta a punta, como lo van a mostrar. La mitad de los demos que se caen es la primera vez que alguien corre el flujo entero.

Dos: **tengan una corrida buena guardada.** Guarden la salida de una corrida que funcione. Si en vivo les da un cuatrocientos veintinueve, enseñen esa y explican que es límite de tasa. Eso no es hacer trampa, es tener un respaldo.

Tres: **sepan cuanto tarda.** Cronometrenlo. Si su flujo tarda cuarenta segundos, tienen que saberlo para llenar esos cuarenta segundos hablando, en vez de quedarse callados mirando la terminal.

Cuatro: **revisen los límites.** Si los subieron para probar, bajenlos. Un bucle en vivo, delante del jurado, con el tope en cien, es una forma muy pública de quedarse sin cuota.

Y cinco: **que el MVP diga lo que no sabe.** Lo que acabamos de hablar. Si le faltó una parte, que la anuncie. Es la diferencia entre "tiene una limitación conocida" y "está mal y no se dieron cuenta".

Si un subagente falla y el orquestador sigue, ¿quién decide si la respuesta final todavía sirve?

La respuesta es: tú, en el prompt del orquestador. Es una decisión de producto, no de código. Si no se lo dices, se la inventa.

105:00 – 117:00

Ejercicio 6 · el esqueleto de tu MVP

Ejercicio seis. Este archivo es suyo y se lo llevan. Lo siguen el sábado.

Primero en papel. Cinco minutos, sin teclado. Y no copien el ejemplo de archivos de los ejercicios: pongan lo que su equipo va a construir de verdad.

Cuatro preguntas. Una: ¿qué hace su MVP, en UNA frase? Si necesitan dos, todavía no está claro. Dos: ¿qué subagentes necesita, y por cada uno, qué herramientas SÍ tiene y qué herramientas NO tiene. Tres: ¿cuál de las cuatro razones justifica separarlo. Cuatro: ¿cuál es su límite de peticiones.

De las cuatro, la que importa es **que herramientas NO tiene**. Si no pueden contestar eso de un subagente, todavia no saben que estan construyendo: saben que quieren uno, que no es lo mismo. Contestar esa pregunta les obliga a decidir el limite de responsabilidad, y de ahi sale el prompt casi solo.

Y aqui esta el filtro que les prometi al principio, cuando les dije que la mitad de la gente usa esto mal. Por cada subagente, marquen: ¿aisla contexto? ¿acota permisos? ¿reduce decisiones? ¿cambia de modelo?

Si no marcaron ninguna, **ese subagente no deberia existir**. Haganlo una herramienta normal del orquestador y ya. Porque multi-agente no es gratis: cada salto son peticiones extra, latencia extra, y una traduccion mas donde se pierde informacion — la que vimos en la ultima tarea del ejercicio tres.

Y una cosa practica, de la que nadie se acuerda hasta que duele: pongan MIS-LIMITES antes de la primera corrida. Cuando lleven tres subagentes, el techo del padre es el unico que cuenta — si pasaron el ctx.

Y hagamos la cuenta de la cuota, porque es la que decide si el sabado llegan al demo o no, y casi nadie la hace.

Tienen entre cinco y quince peticiones por minuto. Una corrida de su MVP, con un orquestador y dos subagentes, les va a costar facil **de seis a diez peticiones**. Hagan la division: eso es **una o dos corridas completas por minuto**, en el mejor caso. No veinte. Una o dos.

O sea que si se ponen a probar cambiando una palabra del prompt y corriendo otra vez, van a estar esperando mas de lo que van a estar programando. Y a las once de la noche, esperando sesenta segundos entre intento e intento, se toman malas decisiones.

Por eso insisto con MODO igual test, y no es un detalle del taller: es la estrategia. **Todo lo que se puede verificar sin el modelo real, verifiquenlo sin el modelo real**. Que las herramientas se llaman, que las delegaciones ocurren, que el esquema valida, que el try-except atrapa. Nada de eso necesita que el modelo piense. Guarden las peticiones de verdad para las preguntas que solo el modelo real puede contestar.

Y una ultima sobre el alcance, que es donde se pierden los hackathones. Su MVP tiene que hacer **una** cosa de punta a punta, no cinco a medias. El sabado nadie les va a dar puntos por la arquitectura: les van a pedir que lo corran. Si corre y resuelve una cosa, ganaron. Si tiene cinco features y dos se caen en el demo, perdieron con mas trabajo.

Asi que cuando contesten la primera pregunta — que hace en una frase — esa frase es su alcance. Todo lo que no quepa ahi es para despues del demo.

ELLOS TRABAJAN — tu te callas. 5 min. Circula. No revises codigo: pregunta "que hace tu MVP en una frase" y "cuantos subagentes y por que". Si no contestan en veinte segundos, ahi esta el trabajo, no en el teclado.

117:00 – 120:00

Cierre

Vamos a cerrar. Dos ideas. Si se les olvida todo lo demas — la sintaxis, los nombres de los parametros, todo — estas dos aguantan.

La primera: **un subagente es una herramienta**, y su descripcion es un prompt. Todo lo del ejercicio uno aplica igual al ejercicio tres. Y de ahi sale el diagnostico mas util que les puedo dar hoy: si el orquestador no esta usando a su subagente, casi siempre el problema no es el subagente. Es el docstring.

La segunda: **lo que garantiza algo es el codigo, no el prompt**. El explorador no escribe porque no tiene la herramienta. Ningun agente se sale del sandbox porque hay una funcion que rechaza la ruta. El padre no se cree al hijo porque hay un esquema y un if. Y la corrida no se muere porque hay un try-except.

Cuando le den herramientas de escritura a un agente — y el sabado se las van a dar — esa es la diferencia que mas les va a importar.

Y una tercera, de regalo: pongan los limites antes de la primera corrida. No despues del susto.

Para el sabado, tres cosas concretas y me callo. Una: **empiecen en MODO igual test**. Armen la forma completa — los agentes, las herramientas, las delegaciones — sin gastar una sola peticion. Cuando corra de punta a punta, entonces cambien a gemini. Van a llegar al demo con cuota.

Dos: **el repo se lo llevan entero**. Taller, config punto py tiene el manejo de errores ya escrito: el cuatrocientos veintinueve, el modelo que no existe, la llave mala, el limite alcanzado, todo traducido a algo que se entiende. Copienlo. No lo escriban otra vez a las dos de la manana.

Y tres: **el ejercicio seis es su plan**. Si el sabado empiezan a programar sin haber contestado las cuatro preguntas, van a construir subagentes que no hacian falta. Contesten primero. Son cinco minutos y les ahorran dos horas.

¿Que van a construir? Una frase por equipo.

Dos o tres equipos, no mas. Cierra con sus palabras, no con un resumen tuyo.